

编 号	题 号	标 题	知 识 点	代 码 作 者
例 3-41	UVa 10341	Solve It	非线性方程求根；牛顿法	刘汝佳
例 3-42	HDU 3714	Error Curves	凸函数求极值；三分法	刘汝佳
例 3-43	UVa 1356	Bridge	数值积分；自适应 Simpson 公式	刘汝佳

3.9 数学专题

本节补充了一些在竞赛中不那么常见，但很有代表性的数学问题及其实现。

例 3-44 【Pick 定理】绿色的世界（A Greener World，UVa 11017）

有一个网格，每个格点中都有一棵梨树。整个网格错切了 θ 度，于是每个格子都变成了一个菱形。现要求在每个菱形的中间重新栽种一颗桃树，如图3-10所示。

给定一个格点多边形，求它的面积和内部的桃树棵数（种在多边形边界上的树不统计在内）。

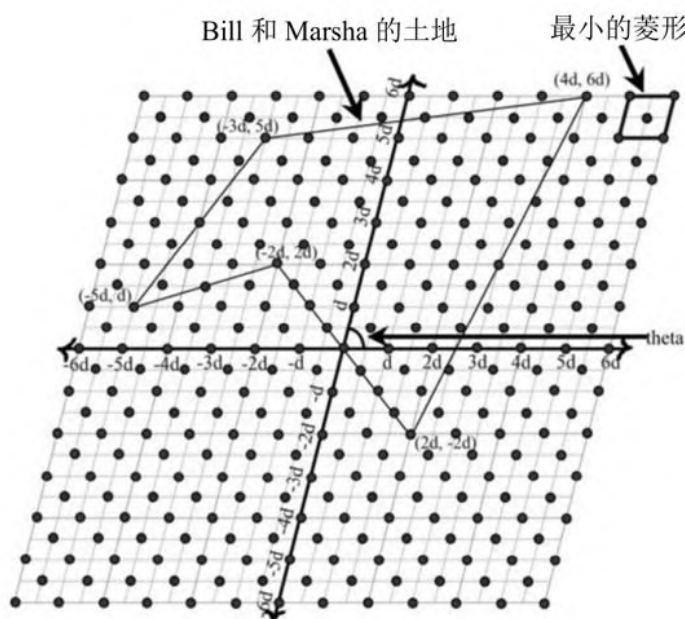


图3-10 绿色的世界

输入包含最多15组数据。每组数据：第一行为3个整数 d , θ 和 N ($1 \leq d < 10\,000$, $44 < \theta < 136$)，其中 N 是多边形的顶点数；以下 N 行，每行包含两个整数 x 和 y ，即多边形的顶点 ($0 \leq x, y \leq 100\,000$)。顶点按照顺时针或逆时针顺序排列。输入结束标志为 $d=\theta=N=0$ 。

【代码实现】

```
// 刘汝佳
#include <cmath>
#include <cstdio>
#include <vector>
using namespace std;

typedef long long LL;

const double PI = acos(-1.0);

struct Point {
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}
};

typedef Point Vector;

Vector operator+(const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator-(const Point& A, const Point& B)
{ return Vector(A.x - B.x, A.y - B.y); }
double Cross(const Vector& A, const Vector& B)
{ return (LL)A.x * B.y - (LL)A.y * B.x; }
```

```

LL PolygonArea2(const vector<Point>& p) {
    int n = p.size();
    LL area2 = 0;
    for (int i = 1; i < n - 1; i++) area2 += Cross(p[i] - p[0], p[i + 1] - p[0]);
    return abs(area2);
}

inline int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }

// 线段 a-b 上的格点数, 不包含 a 和 b。设参数 t = b/d,
// 则 d 必须是 b.x-a.x 和 b.y-a.y 的公约数, 且 0<b<d,
// 减 1 因为要排除端点, 因此 0 和 d 都不能做分子
LL count_on_segment(const Point& a, const Point& b) {
    return gcd(abs(b.x - a.x), abs(b.y - a.y)) - 1;
}

// Pick's Theorem: A = I + B/2 - 1 => I = A - B/2 + 1
LL count_inside_polygon(const vector<Point>& poly) {
    int n = poly.size();
    LL A2 = PolygonArea2(poly);
    int B = n; // 多边形的顶点
    for (int i = 0; i < n; i++) B += count_on_segment(poly[i], poly[(i + 1) % n]);
    return (A2 - B) / 2 + 1;
}

// 计算内部的 x 和 y 的小数部分都是 0.5 的点
LL count(const vector<Point>& poly) {
    vector<Point> poly2;
    for (int i = 0; i < poly.size(); i++) // 旋转 45 度后的稠密网格坐标
        poly2.push_back(Point(poly[i].x - poly[i].y, poly[i].x + poly[i].y));
    return count_inside_polygon(poly2) - count_inside_polygon(poly);
}

int main() {
    // theta 和 d 仅仅用来计算面积
    for (int d, theta, N, x, y; scanf("%d%d%d", &d, &theta, &N) == 3 && d;) {
        vector<Point> poly;
        for (int i = 0; i < N; i++)
            scanf("%d%d", &x, &y), poly.push_back(Point(x, y));
        LL area2 = PolygonArea2(poly);
        printf("%lld %.0lf\n", count(poly),
            sin((double)theta / 180 * PI) * d * d * area2 / 2.0);
    }
    return 0;
}

/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.5.1 节例题 20
同类题目: Area, POJ 1265
*/

```

例3-45 【Lucas定理】有趣的杨辉三角 (Interesting Yang Hui Triangle, POJ 3146)

给出素数 P 和正整数 N ，求杨辉三角形的第 N 行中能整除 P 的数有几个 ($P \leq 1000$, $N \leq 10^9$)。

【代码实现】

```
// 刘汝佳
#include <cstdio>
int main() {
    for (int kase = 0, n, p; scanf("%d%d", &p, &n) == 2 && p;) {
        int ans = 1;
        while (n > 0) ans = ans * (n % p + 1) % 10000, n /= p;
        printf("Case %d: %04d\n", ++kase, ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.5.1 节例题 21
同类题目：Binary Stirling Number, POJ 1430
*/
```

例3-46 【线性规划】食物分配 (Happiness, UVa 10498)

有 n 种食物和 m ($3 \leq n, m \leq 20$) 个人，你的任务是买一些食物，使得每个人都不会吃撑，且在此前提下尽量多花钱。对于每个人 i 来说，每种食物 j 都有一个系数 a_{ij} ，表示单位这种食物为这个人带来的愉悦值。每个人 i 还有一个最大愉悦值 b_i ，表示当食物为他带来的总愉悦值超过 b_i 时，此人将会吃撑。输出可花费钱数的最大值，向上取整。

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <algorithm>
#include <cassert>
using namespace std;

/* 改进单纯形法的实现
参考: http://en.wikipedia.org/wiki/Simplex\_algorithm
输入矩阵 a 描述线性规划的标准形式。a 为 m+1 行 n+1 列, 其中行 0~m-1 为不等式, 行 m 为目标函数(最
大化)。列 0~n-1 为变量 0~n-1 的系数, 列 n 为常数项
第 i 个约束为  $a[i][0]*x[0] + a[i][1]*x[1] + \dots \leq a[i][n]$ 
目标为  $\max(a[m][0]*x[0] + a[m][1]*x[1] + \dots + a[m][n-1]*x[n-1] - a[m][n])$ 
注意: 变量均有非负约束  $x[i] \geq 0$  */
const int maxm = 500;           // 约束数目上限
const int maxn = 500;           // 变量数目上限
const double INF = 1e100, eps = 1e-10;

struct Simplex {
    int n;                       // 变量个数

```

```

int m; // 约束个数
double a[maxm][maxn]; // 输入矩阵
int B[maxm], N[maxn]; // 算法辅助变量

void pivot(int r, int c) {
    swap(N[c], B[r]);
    a[r][c] = 1 / a[r][c];
    for (int j = 0; j <= n; j++) if (j != c) a[r][j] *= a[r][c];
    for (int i = 0; i <= m; i++) if (i != r) {
        for (int j = 0; j <= n; j++) if (j != c) a[i][j] -= a[i][c] * a[r][j];
        a[i][c] = -a[i][c] * a[r][c];
    }
}

bool feasible() {
    for (;;) {
        int r, c;
        double p = INF;
        for (int i = 0; i < m; i++) if (a[i][n] < p) p = a[r = i][n];
        if (p > -eps) return true;
        p = 0;
        for (int i = 0; i < n; i++) if (a[r][i] < p) p = a[r][c = i];
        if (p > -eps) return false;
        p = a[r][n] / a[r][c];
        for (int i = r + 1; i < m; i++) if (a[i][c] > eps) {
            double v = a[i][n] / a[i][c];
            if (v < p) { r = i; p = v; }
        }
        pivot(r, c);
    }
}

// 解有界返回1, 无解返回0, 无界返回-1. b[i]为x[i]的值, ret为目标函数的值
int simplex(int n, int m, double x[maxn], double& ret) {
    this->n = n;
    this->m = m;
    for (int i = 0; i < n; i++) N[i] = i;
    for (int i = 0; i < m; i++) B[i] = n + i;
    if (!feasible()) return 0;
    for (;;) {
        int r, c;
        double p = 0;
        for (int i = 0; i < n; i++) if (a[m][i] > p) p = a[m][c = i];
        if (p < eps) {
            for (int i = 0; i < n; i++) if (N[i] < n) x[N[i]] = 0;
            for (int i = 0; i < m; i++) if (B[i] < n) x[B[i]] = a[i][n];
            ret = -a[m][n];
            return 1;
        }
    }
}

```

```

    p = INF;
    for (int i = 0; i < m; i++) if (a[i][c] > eps) {
        double v = a[i][n] / a[i][c];
        if (v < p) { r = i; p = v; }
    }
    if (p == INF) return -1;
    pivot(r, c);
}
}
};

////////// 题目相关
#include <cmath>
Simplex solver;

int main() {
    for (int n, m; scanf("%d%d", &n, &m) == 2;) {
        for (int i = 0; i < n; i++) scanf("%lf", &solver.a[m][i]); // 目标函数
        solver.a[m][n] = 0; // 目标函数常数项
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n + 1; j++)
                scanf("%lf", &solver.a[i][j]);
        double ans, x[maxn];
        assert(solver.simplex(n, m, x, ans) == 1);
        ans *= m;
        printf("Nasa can spend %d taka.\n", (int)floor(ans + 1 - eps));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.5.2 节例题 23
同类题目：Mission Possible, HDU 5931
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表3-9所示。

表3-9 数学专题例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 3-44	UVa 11017	A Greener World	Pick 定理	刘汝佳
例 3-45	POJ 3146	Interesting Yang Hui Triangle	Lucas 定理	刘汝佳
例 3-46	UVa 10498	Happiness	线性规划	刘汝佳